

1. Database & Collection Setup:

- o Create a new database (e.g., studentDB)
- o Create a collection (e.g., students)

```
(c) Microsoft Corporation. All rights reserved.

C:\WINDOWS\system32>mongosh
Current Mongosh Log ID: 69f61d74361033a4473682d0
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.8.2
Using MongoDB:      8.2.5
Using Mongosh:       2.8.2

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----
  The server generated these startup warnings when booting
  2026-05-01T07:21:46.663+05:30: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
-----

test> use studentDB
switched to db studentDB
studentDB> db.createcollection("students")
TypeError: db.createcollection is not a function
studentDB> db.createCollection("students")
{ ok: 1 }
studentDB> db.student.insertOne({name:"FENNNJOY",age:21,course:"MEAN Stact",status:"ongoing"})
{
  acknowledged: true,
  insertedId: ObjectId('69f62058361033a4473682d1')
}
studentDB>
```

2. Insert Operations:

- o Insert one document into the students collection.
- o Insert multiple documents with fields like name, age, course, status.

```
studentDB> db.student.insertOne({name:"alan",age:21,course:"python",status:"ongoing"})
{
  acknowledged: true,
  insertedId: ObjectId('69f62925361033a4473682d2')
}
studentDB> db.student.insertMany([{name:"alex",age:19,course:"c++",status:"ongoing"},{name:"jack",age:22,course:"Java",status:"ongoing"}])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69f62b53361033a4473682d3'),
    '1': ObjectId('69f62b53361033a4473682d4')
  }
}
studentDB> _
```

3. Read Operations:

- o Fetch all documents from the collection.
- o Use filters to fetch specific data (e.g., students enrolled in “MERN Stack”).

```
]
studentDB> db.student.find()
[
  {
    _id: ObjectId('69f62058361033a4473682d1'),
    name: 'FENNJOY',
    age: 21,
    course: 'MEAN Stact',
    status: 'ongoing'
  },
  {
    _id: ObjectId('69f62925361033a4473682d2'),
    name: 'alan',
    age: 21,
    course: 'python',
    status: 'ongoing'
  },
  {
    _id: ObjectId('69f62b53361033a4473682d3'),
    name: 'alex',
    age: 19,
    course: 'c++',
    status: 'ongoing'
  },
  {
    _id: ObjectId('69f62b53361033a4473682d4'),
    name: 'jack',
    age: 22,
    course: 'Java',
    status: 'ongoing'
  }
]
studentDB>
```

```
studentDB> db.student.find({course:"Java"})
[
  {
    _id: ObjectId('69f62b53361033a4473682d4'),
    name: 'jack',
    age: 22,
    course: 'Java',
    status: 'ongoing'
  }
]
studentDB>
```

4. Update Operations:

- o Update a single document (e.g., change the status of a student to completed).
- o Update multiple documents at once.

```
[
  {
    _id: ObjectId('69f62b53361033a4473682d4'),
    name: 'jack',
    age: 22,
    course: 'Java',
    status: 'ongoing'
  }
]
studentDB> db.student.updateOne({name:'jack'},{$set:{status:'completed'}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
studentDB> db.student.find({name:"jack"})
[
  {
    _id: ObjectId('69f62b53361033a4473682d4'),
    name: 'jack',
    age: 22,
    course: 'Java',
    status: 'completed'
  }
]
studentDB>
```

```
studentDB> db.student.updateMany({status:"ongoing"},{$set:{status:"completed"}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 0,
  modifiedCount: 0,
  upsertedCount: 0
}
```

```
studentDB> db.student.find({status:"completed"})
[
  {
    _id: ObjectId('69f62058361033a4473682d1'),
    name: 'FENNJOY',
    age: 21,
    course: 'MEAN Stact',
    status: 'completed'
  },
  {
    _id: ObjectId('69f62925361033a4473682d2'),
    name: 'alan',
    age: 21,
    course: 'python',
    status: 'completed'
  },
  {
    _id: ObjectId('69f62b53361033a4473682d3'),
    name: 'alex',
    age: 19,
    course: 'c++',
    status: 'completed'
  },
  {
    _id: ObjectId('69f62b53361033a4473682d4'),
    name: 'jack',
    age: 22,
    course: 'Java',
    status: 'completed'
  }
]
```

5. Delete Operations:

- o Delete one document based on a condition.
- o Delete all documents in the collection (for practice purpose only).

```
studentDB> db.student.updateMany({status:"ongoing"},{$set:{status:"completed"}})

studentDB> db.student.deleteOne({name:"FENNJOY"})
{ acknowledged: true, deletedCount: 1 }
```

```
studentDB> db.student.find()
[
  {
    _id: ObjectId('69f62925361033a4473682d2'),
    name: 'alan',
    age: 21,
    course: 'python',
    status: 'completed'
  },
  {
    _id: ObjectId('69f62b53361033a4473682d3'),
    name: 'alex',
    age: 19,
    course: 'c++',
    status: 'completed'
  },
  {
    _id: ObjectId('69f62b53361033a4473682d4'),
    name: 'jack',
    age: 22,
    course: 'Java',
    status: 'completed'
  }
]
```

```
studentDB> db.student.updateMany({status:"ongoing"},{$set:{status:"completed"}})

studentDB> db.student.deleteMany({})
{ acknowledged: true, deletedCount: 3 }
studentDB> db.student.find()

studentDB>
```

6. Query Operators:

- o Practice using \$gt, \$lt, \$in, \$and, \$or, \$exists in different queries.

```

studentDB> db.student.insertMany([{"name":"fennjoy",age:21,course:"MERN STACK",status:"ongoing"},{"name":"alex",age:22,course:"c++",status:"completed"},{"name":"alan",age:24,course:"python",status:"ongoing"}])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69f63e11361033a4473682d5'),
    '1': ObjectId('69f63e11361033a4473682d6'),
    '2': ObjectId('69f63e11361033a4473682d7')
  }
}
studentDB> db.student.find({age:{ $gt:21}})
[
  {
    _id: ObjectId('69f63e11361033a4473682d6'),
    name: 'alex',
    age: 22,
    course: 'c++',
    status: 'completed'
  },
  {
    _id: ObjectId('69f63e11361033a4473682d7'),
    name: 'alan',
    age: 24,
    course: 'python',
    status: 'ongoing'
  }
]
studentDB>

```

Activate Windows
Go to Settings to activate Windows.

```

studentDB> db.student.find({age:{ $lt:24}})
[
  {
    _id: ObjectId('69f63e11361033a4473682d5'),
    name: 'fennjoy',
    age: 21,
    course: 'MERN STACK',
    status: 'ongoing'
  },
  {
    _id: ObjectId('69f63e11361033a4473682d6'),
    name: 'alex',
    age: 22,
    course: 'c++',
    status: 'completed'
  }
]
studentDB>

```

```
]
studentDB> db.student.find({course:{$in:["MERN STACK","python"]}})
[
  {
    _id: ObjectId('69f63e11361033a4473682d5'),
    name: 'fennjoy',
    age: 21,
    course: 'MERN STACK',
    status: 'ongoing'
  },
  {
    _id: ObjectId('69f63e11361033a4473682d7'),
    name: 'alan',
    age: 24,
    course: 'python',
    status: 'ongoing'
  }
]
studentDB> _
```

```
]
studentDB> db.student.find({$and:[{age:22},{status:"completed"}]})
[
  {
    _id: ObjectId('69f63e11361033a4473682d6'),
    name: 'alex',
    age: 22,
    course: 'c++',
    status: 'completed'
  }
]
studentDB>
```



```
studentDB> db.student.find({$or:[{course:"c++"},{course:"python"}]})
[
  {
    _id: ObjectId('69f63e11361033a4473682d6'),
    name: 'alex',
    age: 22,
    course: 'c++',
    status: 'completed'
  },
  {
    _id: ObjectId('69f63e11361033a4473682d7'),
    name: 'alan',
    age: 24,
    course: 'python',
    status: 'ongoing'
  }
]
studentDB> _
```

```

]
studentDB> db.student.find({age:{$exists:true}})
[
  {
    _id: ObjectId('69f63e11361033a4473682d5'),
    name: 'fennjoy',
    age: 21,
    course: 'MERN STACK',
    status: 'ongoing'
  },
  {
    _id: ObjectId('69f63e11361033a4473682d6'),
    name: 'alex',
    age: 22,
    course: 'c++',
    status: 'completed'
  },
  {
    _id: ObjectId('69f63e11361033a4473682d7'),
    name: 'alan',
    age: 24,
    course: 'python',
    status: 'ongoing'
  }
]
studentDB>

```

7. Use Case:

o Create a small use case like a Library System or E-Commerce product listing and apply insert, update, and search operations.

Outcome:

```
PS C:\Users\fennj\OneDrive\Desktop\g> mongosh
mongosh 2.8.3 is available for download: https://www.mongodb.com/try/download/shell
```

```
For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/
```

```
-----
The server generated these startup warnings when booting
2026-05-01T07:21:46.663+05:30: Access control is not enabled for the database.
Read and write access to data and configuration is unrestricted
-----
```

```
test> use libraryDB
switched to db libraryDB
libraryDB> db.books.insertMany([
|   { title: "Book A", author: "Author 1", available: true },
|   { title: "Book B", author: "Author 2", available: false },
|   { title: "Book C", author: "Author 3", available: true }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69f6476151657f26813682d1'),
    '1': ObjectId('69f6476151657f26813682d2'),
    '2': ObjectId('69f6476151657f26813682d3')
  }
}
```

```
libraryDB> db.books.find({ available: true })
[
  {
    _id: ObjectId('69f6476151657f26813682d1'),
    title: 'Book A',
    author: 'Author 1',
    available: true
  },
  {
    _id: ObjectId('69f6476151657f26813682d3'),
    title: 'Book C',
    author: 'Author 3',
    available: true
  }
]
libraryDB> db.books.updateOne(
|   { title: "Book A" },
|   { $set: { available: false } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

```
}  
libraryDB> db.books.find()  
[  
  {  
    _id: ObjectId('69f6476151657f26813682d1'),  
    title: 'Book A',  
    author: 'Author 1',  
    available: false  
  },  
  {  
    _id: ObjectId('69f6476151657f26813682d2'),  
    title: 'Book B',  
    author: 'Author 2',  
    available: false  
  },  
  {  
    _id: ObjectId('69f6476151657f26813682d3'),  
    title: 'Book C',  
    author: 'Author 3',  
    available: true  
  }  
]  
libraryDB> 
```